

04 - Validaciones y códigos de error

Ejercicio 04 — Validaciones y códigos de error



Foco: validación de input, manejo de errores, patrones repetidos **Tiempo estimado:** 1–2 clases [← Volver al README](#)

Objetivo

Que uses la IA para **detectar un patrón de validación ausente o repetido** y proponer una forma consistente de manejar input inválido y códigos de error. Hoy el proyecto **casi no valida nada**.

El problema

Mirá `alumnos-controller.js` y `alumnos-service.js`:

- En el `POST`, `let entity = req.body` pasa **directo** al service. Si mando `{}` (body vacío), o `nombre` con 5000 caracteres, o `id_curso` como `"hola"`, nadie lo frena hasta que explota la query.
- El `repository` hace `?? ''` y `?? 0` → **silenciosamente** te inserta un alumno con `nombre = ''`. No es un error, pero seguramente no es lo que querés.
- Los errores se manejan con `try/catch` que mayormente devuelve `400` o `500` con un `error.message` crudo (que puede filtrar info de la base — ojo, esto se cruza con el ejercicio 09).
- La única validación de negocio que existe es `validarCursoExiste` en el service. **Está buenísima como modelo a seguir**, pero es la excepción, no la regla.
- El **id de la URL se trata distinto en cada endpoint**: el `GET /:id` y el `DELETE /:id` lo usan como viene (un string), pero el `PUT /:id` hace `parseInt`. El criterio quedó inconsistente, y nadie valida que el `id` sea realmente un número.

El caso del `id`: el mismo dato, validado de tres formas distintas

Abrí los controllers y compará los tres endpoints que reciben un `id`:

```
// GET /:id y DELETE /:id
let id = req.params.id;           // queda STRING, va directo al SQL (WHERE id=$1)
```

```
// PUT /:id
```

```
let id = parseInt(req.params.id); // Lo convierte a NÚMERO para compararlo con el id del body
```

No es casualidad: el PUT necesita el número porque **compara en JavaScript** (`parseInt(entity.id) !== id`), mientras que el GET/DELETE solo se lo pasan a `pg` (que castea solo). **Funciona**, pero el criterio quedó **inconsistente** y nadie valida que el `id` sea realmente un número → `GET /api/alumnos/abc` hoy te puede tirar un **500 críptico** en vez de un error claro.

🔗 **La oportunidad:** extraé un helper que valide y convierta el `id` en un solo lugar (algo del estilo `parsearId`, en `src/helpers/validaciones-helper.js`) y usalo **igual en los tres endpoints**. Así el criterio queda unificado y un `id` inválido devuelve un código sensato en vez de explotar.

El caso más rico: calificaciones

Si en el ejercicio 01 creaste la tabla `calificaciones`, es **el caso ideal** para practicar acá, porque junta varios tipos de validación a la vez:

- **Rango:** la `nota` tiene que ser un entero entre 0 y 10. ¿Qué pasa hoy con `nota: 99` o `nota: "diez"`?
- **Existencia de FKs:** `id_alumno` e `id_materia` tienen que existir → se valida **reutilizando los services** (igual que `validarCursoExiste` usa `CursosService`).
- **Conflicto (409):** no puede haber dos calificaciones del mismo alumno en la misma materia (la tabla tiene `UNIQUE(id_alumno, id_materia)`). Si ya existe, eso es un **409 Conflict** — no un 400, no un 500.

💡 No te decimos *cómo* validar cada cosa —eso lo resolvés vos con la IA—, solo *qué* casos tenés que cubrir.

📦 Qué tenés que lograr

1. Definir **qué es un alumno válido** (¿ `nombre` obligatorio? ¿ `fecha_nacimiento` con formato válido? ¿ `id_curso` numérico? ¿ `hace_deportes` booleano?).
2. Implementar validación de input **antes** de llegar al repository. Elegí dónde: middleware, capa `business/`, o helper de validación (justificá).
3. Crear un **helper que valide y convierta el id** (p. ej. `parsearId` en `src/helpers/validaciones-helper.js`) y usarlo en `GET /:id`, `PUT /:id` y `DELETE /:id`, **unificando** el criterio que hoy está inconsistente. Un `id` no numérico tiene que devolver un código claro (**400**), no un 500.
4. Unificar los **códigos de error**: 400 para input inválido, 404 para no encontrado, 409 para conflicto, 500 para error inesperado. Que sea **consistente** entre entidades.
5. Devolver mensajes de error **útiles para el cliente pero que no filtren detalles internos** de la base.
6. Si hiciste `calificaciones` en el ejercicio 01, aplicá ahí las validaciones de arriba (rango de `nota`, existencia de FKs reutilizando services, y **409** por duplicado). Es el caso que mejor ejercita los 4 códigos de error.

🤖 Cómo encarar el prompting

💡 Este ejercicio es ideal para que la IA primero **audite** y vos decidas. El prompt de diagnóstico es el más valioso.

Prompt 1 — Auditoría de validación:

Pegale el controller, el service y el repository de `alumnos`, y pedile: *"hacé una tabla de todos los inputs que el usuario puede mandar y qué pasa hoy si cada uno viene vacío, con tipo incorrecto, o malicioso. No escribas código todavía — quiero ver los agujeros."*

Prompt 2 — Diseño del patrón:

"Proponé un patrón de validación consistente para esta arquitectura en capas. ¿Validación en middleware, en el service, o con una librería? Dame pros y contras de cada uno para un proyecto educativo sin dependencias pesadas."

Prompt 3 — Implementación.

⚠️ **Trampa de los códigos de error:** la IA tiende a devolver `500` para todo lo que falla. Pero un body inválido es **culpa del cliente** → eso es `400`, no `500`. Pedile explícitamente que **distinga error del cliente (4xx) de error del servidor (5xx)**.

⚠️ **Otra trampa:** si le pedís "validá todo", te puede meter `zod`, `joi` o `express-validator`. Eso puede estar bien, pero es una **decisión de arquitectura** — no la tomes por default. Si agregás una dependencia, justificala en la reflexión.

🔍 Verificación del resultado

- ☐ `POST /api/alumnos` con body `{}` devuelve **400** (no 500, no un 201 con campos vacíos).
- ☐ `POST` con `id_curso` que no existe sigue devolviendo el error de negocio correcto (no rompas `validarCursoExiste`).
- ☐ `GET /api/alumnos/abc` (id no numérico) devuelve **400** con un mensaje claro, no un 500 críptico.
- ☐ El `id` se valida y convierte con **un solo helper** (`parsearId`), usado en `GET`, `PUT` y `DELETE` — ya no hay un endpoint con `parseInt` y otro sin él.
- ☐ Los mensajes de error **no incluyen** el texto crudo del error de PostgreSQL (nombres de tablas, columnas, etc.).
- ☐ La validación es **consistente**: `alumnos` y `cursos` validan con el mismo patrón.
- ☐ Mostrás que entendés **dónde** pusiste la validación y por qué (no "porque la IA la puso ahí").

🗣️ Pregunta para el oral: *¿la validación va en el controller, en el service o en un middleware? Defendé tu elección. ¿Qué pasa si la misma regla la necesitás en dos endpoints distintos?*

🗣️ Pregunta para el oral (el `id`): *antes, ¿por qué el PUT hacía `parseInt` y el GET no? ¿Tu `parsearId` devuelve el número o tira un error? Y un `id` no numérico, ¿es 400 o 404? Defendé tu elección.*

✅ Entrega

Bitácora + commit. **Caso obligatorio en la reflexión:** mostrará un input inválido concreto, qué hacía el sistema antes (lo dejaba pasar / tiraba 500) y qué hace ahora.